

Bài tập Thuật toán và Thiết kế thuật toán

Vu Tien Dat

Ngày 17 tháng 4 năm 2026

Câu 2: Bài toán qua cầu (Bridge and Torch Problem)

Đề bài

Có 4 người cần băng qua một cây cầu hẹp vào ban đêm. Họ chỉ có duy nhất một chiếc đèn pin. Tối đa hai người có thể cùng qua cầu một lúc. Bất kỳ nhóm nào qua cầu (dù là một hay hai người) đều phải mang theo đèn pin. Đèn pin phải được cầm đi đi lại lại (không được ném). Thời gian đi của mỗi người:

- Người 1: 1 phút
- Người 2: 2 phút
- Người 3: 5 phút
- Người 4: 10 phút

Khi hai người đi cùng nhau, thời gian qua cầu được tính theo người đi chậm hơn.

Yêu cầu

Tìm phương án đưa cả 4 người qua cầu với **tổng thời gian nhỏ nhất**.

Lời giải chi tiết

Chiến lược tối ưu

Bài toán này có lời giải nổi tiếng dựa trên hai chiến lược chính, và chiến lược tối ưu thường kết hợp cả hai:

1. **Chiến lược A (Fast shuttle):** Hai người nhanh nhất (1 và 2) qua cầu trước, người nhanh nhất (1) cầm đèn quay lại, sau đó hai người chậm nhất (5 và 10) qua cùng nhau, cuối cùng người nhanh thứ hai (2) quay lại đón người còn lại.
2. **Chiến lược B (Paired slow):** Người nhanh nhất (1) đưa lần lượt từng người chậm qua cầu, mỗi lần đều quay lại.

Tính toán cụ thể

Phương án 1 (Chiến lược A):

- 1 và 2 qua cầu: mất $\max(1, 2) = 2$ phút. (Tổng: 2 phút)
- 1 quay lại: mất 1 phút. (Tổng: 3 phút)
- 5 và 10 qua cầu: mất $\max(5, 10) = 10$ phút. (Tổng: 13 phút)
- 2 quay lại: mất 2 phút. (Tổng: 15 phút)
- 1 và 2 qua cầu: mất 2 phút. (Tổng: **17 phút**)

Phương án 2 (Chiến lược B):

- 1 và 10 qua cầu: mất 10 phút. (Tổng: 10 phút)
- 1 quay lại: mất 1 phút. (Tổng: 11 phút)
- 1 và 5 qua cầu: mất 5 phút. (Tổng: 16 phút)
- 1 quay lại: mất 1 phút. (Tổng: 17 phút)
- 1 và 2 qua cầu: mất 2 phút. (Tổng: **19 phút**)

So sánh và kết luận

- Phương án 1: 17 phút
- Phương án 2: 19 phút

Phương án 1 nhanh hơn, nhưng liệu có phương án nào tối ưu hơn không?

Phương án tối ưu thực sự (kết hợp cả hai chiến lược)

Đây là phương án chuẩn được nhiều tài liệu công nhận:

Bước 1: 1 và 2 qua cầu (mất 2 phút). Đèn bên kia cầu. Tổng: 2 phút.

Bước 2: 1 quay lại (mất 1 phút). Tổng: 3 phút.

Bước 3: 5 và 10 qua cầu (mất 10 phút). Đèn bên kia cầu. Tổng: 13 phút.

Bước 4: 2 quay lại (mất 2 phút). Tổng: 15 phút.

Bước 5: 1 và 2 qua cầu (mất 2 phút). Tổng: 17 phút.

Kiểm chứng bằng tư duy thuật toán

Gọi 4 người có thời gian lần lượt là $a \leq b \leq c \leq d$ (ở đây $a = 1, b = 2, c = 5, d = 10$). Thuật toán tối ưu tổng quát so sánh hai cách đưa hai người chậm nhất c, d qua cầu:

- Cách 1: a, b qua $\rightarrow a$ về $\rightarrow c, d$ qua $\rightarrow b$ về. Chi phí cho cặp c, d : $b+a+d+b = 2b+a+d$.
- Cách 2: a, d qua $\rightarrow a$ về $\rightarrow a, c$ qua $\rightarrow a$ về. Chi phí: $d+a+c+a = 2a+c+d$.

Chọn cách có chi phí nhỏ hơn. Với $a = 1, b = 2, c = 5, d = 10$:

- Cách 1: $2b+a+d = 4+1+10 = 15$
- Cách 2: $2a+c+d = 2+5+10 = 17$

Chọn cách 1 cho cặp (5,10), cộng thêm bước cuối đưa a, b qua (2 phút) và trừ đi bước a về thừa, ta được 17 phút.

Vậy thời gian tối thiểu là **17 phút**.

Trả lời phỏng vấn

Nếu được nhà tuyển dụng hỏi, câu trả lời ngắn gọn: “Phương án tối ưu là 17 phút, với thứ tự: 1&2 qua $\rightarrow$ 1 về $\rightarrow$ 5&10 qua $\rightarrow$ 2 về $\rightarrow$ 1&2 qua.”

Câu 3: Công thức tính diện tích tam giác khi biết ba cạnh

Đề bài đưa ra ba công thức và hỏi công thức nào có thể được coi là **một thuật toán** để tính diện tích tam giác khi biết độ dài ba cạnh $a, b, c > 0$.

Phân tích từng công thức

a. $S = \sqrt{p(p-a)(p-b)(p-c)}$, với $p = \frac{a+b+c}{2}$ (Công thức Heron)

- **Đầu vào:** a, b, c là ba cạnh tam giác.
- **Các bước:**
 1. Tính $p = (a+b+c)/2$.
 2. Tính $p-a, p-b, p-c$.
 3. Tính tích $p \cdot (p-a) \cdot (p-b) \cdot (p-c)$.
 4. Tính căn bậc hai của kết quả.
- **Đặc điểm:** Chỉ sử dụng các phép tính số học cơ bản (cộng, trừ, nhân, chia, khai căn). Không cần thêm thông tin nào khác.
- **Kết luận:** Đây là một thuật toán rõ ràng, đầy đủ, và được dạy rộng rãi.

b. $S = \frac{1}{2}bc \sin A$, với A là góc giữa hai cạnh b và c

- **Vấn đề:** Đầu vào chỉ có a, b, c , không có góc A . Để tính $\sin A$, cần biết A hoặc tính A từ a, b, c bằng định lý cos: $\cos A = \frac{b^2 + c^2 - a^2}{2bc}$, rồi $\sin A = \sqrt{1 - \cos^2 A}$.
- Như vậy, công thức này không phải là thuật toán trực tiếp với đầu vào a, b, c ; nó đòi hỏi thêm bước tính góc.
- Nếu bổ sung các bước tính $\cos A$ và $\sin A$, nó trở thành thuật toán nhưng dài hơn và kém hiệu quả hơn Heron.

c. $S = \frac{1}{2}ah_a$, với h_a là chiều cao tương ứng với cạnh đáy a

- **Vấn đề:** Đầu vào chỉ có a, b, c , không có h_a . Để tính h_a , cần dùng công thức $h_a = \frac{2S}{a}$ – vòng lặp vô hạn (cần S để tính h_a , cần h_a để tính S).
- Nếu tính h_a qua công thức $h_a = b \sin C$ hoặc dùng Heron trước, thì công thức này không độc lập.
- Vì vậy, với chỉ a, b, c , công thức này không thể thực thi được.

Kết luận

Chỉ có **công thức a (Heron)** là một thuật toán hoàn chỉnh để tính diện tích tam giác khi chỉ biết ba cạnh a, b, c .

Câu 4: Kỹ thuật thiết kế thuật toán, mã giả và chứng minh tính đúng đắn

Kỹ thuật thiết kế thuật toán (Algorithm design techniques)

Là các phương pháp, mô hình tổng quát, có cấu trúc và có thể tái sử dụng để xây dựng thuật toán cho nhiều bài toán khác nhau. Một số kỹ thuật phổ biến:

- **Chia để trị (Divide and Conquer):** Chia bài toán lớn thành các bài toán con, giải chúng đệ quy, rồi kết hợp kết quả. (VD: Sắp xếp trộn - Merge Sort)
- **Tham lam (Greedy):** Ở mỗi bước, chọn phương án tối ưu cục bộ với hy vọng đạt tối ưu toàn cục. (VD: Bài toán trả tiền xu, cây khung nhỏ nhất Kruskal/Prim)
- **Quy hoạch động (Dynamic Programming):** Chia bài toán thành các bài toán con chồng lấn, lưu kết quả để tránh tính lại. (VD: Bài toán xếp ba lô, đường đi ngắn nhất Floyd–Warshall)
- **Quay lui (Backtracking):** Duyệt toàn bộ không gian lời giải, quay lại khi gặp ngõ cụt. (VD: Bài toán 8 hậu, giải Sudoku)
- **Nhánh và cận (Branch and Bound):** Cải tiến của quay lui, thêm đánh giá cận để cắt bớt nhánh không triển vọng.

Mã giả (Pseudocode)

Là một cách mô tả thuật toán bằng ngôn ngữ lai giữa ngôn ngữ tự nhiên và các cấu trúc lập trình (gán, rẽ nhánh, lặp, gọi hàm), nhưng **không tuân thủ chặt chẽ cú pháp** của bất kỳ ngôn ngữ lập trình cụ thể nào.

Ví dụ mã giả cho thuật toán tìm max:

```
Algorithm TimMax(a1, a2, ..., an)
Input: Dãy số a1..an
Output: Giá trị lớn nhất
    max = a1
    for i = 2 to n do
        if a[i] > max then
            max = a[i]
        end if
    end for
    return max
```

Chứng minh tính đúng đắn của thuật toán

Để chứng minh một thuật toán **luôn trả ra kết quả đúng** cho mọi bộ đầu vào hợp lệ, thường dùng các phương pháp:

1. Quy nạp toán học (Induction)

Áp dụng cho thuật toán có tính lặp hoặc đệ quy. Gồm:

- **Bước cơ sở:** Chứng minh thuật toán đúng với trường hợp nhỏ nhất.
- **Bước quy nạp:** Giả sử thuật toán đúng với bài toán kích thước n , chứng minh đúng với kích thước $n + 1$.

2. Bất biến vòng lặp (Loop invariant)

- Phát biểu một điều kiện luôn đúng **trước** và **sau** mỗi vòng lặp.
- **Khởi tạo:** Điều kiện đúng trước vòng lặp đầu tiên.
- **Duy trì:** Nếu đúng trước một bước lặp, nó vẫn đúng sau bước lặp đó.
- **Kết thúc:** Khi vòng lặp kết thúc, bất biến cùng với điều kiện kết thúc cho ta kết quả đúng.

3. Chứng minh bằng phản chứng (Contradiction)

Giả sử thuật toán sai với một đầu vào, sau đó suy ra mâu thuẫn với giả thiết hoặc với định nghĩa của bài toán.

4. Chứng minh tính dừng (Termination)

Cần chỉ ra rằng thuật toán không rơi vào vòng lặp vô hạn. Thường dùng *hàm năng lượng* (*potential function*) giảm dần sau mỗi bước và bị chặn dưới.

Ví dụ: Chứng minh thuật toán tìm max bằng bất biến vòng lặp

1. **Bất biến:** Sau k bước lặp, max luôn là giá trị lớn nhất trong k phần tử đầu tiên.
2. **Khởi tạo:** $k = 1$, $max = a_1$, hiển nhiên đúng.
3. **Duy trì:** Giả sử đúng sau bước k , xét bước $k + 1$: so sánh a_{k+1} với max , nếu lớn hơn thì cập nhật max . Do đó max mới vẫn là lớn nhất trong $k + 1$ phần tử.
4. **Kết thúc:** Khi $k = n$, max là lớn nhất trong n phần tử $\rightarrow$ kết quả đúng.

Kết luận chung

Một thuật toán được coi là đúng nếu nó dừng sau hữu hạn bước và cho kết quả đúng với mọi đầu vào. Để đảm bảo điều này, người thiết kế cần kết hợp chặt chẽ kỹ thuật thiết kế, mô tả bằng mã giả, và chứng minh tính đúng đắn bằng các phương pháp hình thức.

Câu 5: Thuật toán chuyển đổi số nhị phân sang thập phân

Yêu cầu

Mô tả thuật toán để tìm biểu diễn thập phân của một số nhị phân dương.

a. Mô tả bằng tiếng Anh (ngôn ngữ tự nhiên)

English description:

*The algorithm takes a binary string (e.g., "1011") as input and computes its decimal equivalent. Starting from the leftmost digit, we initialize a result variable to 0. For each binary digit from left to right, we double the current result (shift left in binary) and then add the current digit (0 or 1). After processing all digits, the result variable holds the decimal value. For example, "1011" is processed as: start 0 $\rightarrow 0*2+1=1 \rightarrow 1*2+0=2 \rightarrow 2*2+1=5 \rightarrow 5*2+1=11$.*

Mô tả bằng tiếng Việt:

Thuật toán nhận đầu vào là một chuỗi nhị phân (ví dụ "1011") và tính giá trị thập phân tương ứng. Bắt đầu từ trái sang phải, khởi tạo biến kết quả bằng 0. Với mỗi chữ số nhị phân từ trái qua phải, ta nhân đôi kết quả hiện tại (tương đương dịch trái trong hệ nhị phân) rồi cộng thêm chữ số đó (0 hoặc 1). Sau khi xử lý hết các chữ số, biến kết quả chính là giá trị thập phân cần tìm.

b. Mô tả bằng mã giả (Pseudocode)

Algorithm BinaryToDecimal(binaryString)

Input: binaryString - a string of '0' and '1' characters (e.g., "1011")

Output: decimalValue - the decimal representation

```

decimalValue = 0
for i = 0 to length(binaryString) - 1 do
    digit = integer value of binaryString[i]    // '0' -> 0, '1' -
> 1
    decimalValue = decimalValue * 2 + digit
end for
return decimalValue

```

Ví dụ minh họa:

Với đầu vào "1011":

- Khởi tạo: $decimal = 0$
- $i = 0$, digit = 1: $decimal = 0 \times 2 + 1 = 1$
- $i = 1$, digit = 0: $decimal = 1 \times 2 + 0 = 2$
- $i = 2$, digit = 1: $decimal = 2 \times 2 + 1 = 5$
- $i = 3$, digit = 1: $decimal = 5 \times 2 + 1 = 11$

Kết quả: 11.

Câu 6: Thuật toán thực hiện cuộc gọi trên điện thoại

Mô tả bằng mã giả kết hợp ngôn ngữ tự nhiên

Điện thoại di động sử dụng một hệ thống phức tạp gồm nhiều lớp giao thức. Dưới đây là thuật toán bậc cao (high-level) mô tả quá trình thực hiện một cuộc gọi:

Algorithm MakePhoneCall(phoneNumber, networkProvider)

Input: phoneNumber – số cần gọi (có thể là số nội hạt, liên tỉnh, quốc tế)
networkProvider – nhà mạng (có thể ngầm xác định qua SIM)

Output: Cuộc gọi được thiết lập hoặc thông báo lỗi

```

// Bước 1: Kiểm tra tính hợp lệ
if not isValidPhoneNumber(phoneNumber) then
    return "Số điện thoại không hợp lệ"
end if

```

```

// Bước 2: Kiểm tra tín hiệu mạng
if signalStrength == 0 then
    return "Không có sóng, không thể thực hiện cuộc gọi"
end if

```

```

// Bước 3: Gửi yêu cầu thiết lập cuộc gọi đến trạm BTS gần nhất
callRequest = createCallRequest(phoneNumber, myPhoneID, myLocation)
sendToBaseStation(callRequest)

```

```

// Bước 4: Trạm BTS chuyển tiếp đến tổng đài (MSC – Mobile Switching C

```

```

// Hệ thống mạng xác định tuyến đường đến thuê bao đích

// Bước 5: Gửi tín hiệu nhắc máy (ringing signal) đến thuê bao đích
if destinationBusy() then
    return "Máy bận, thử lại sau"
end if

if destinationUnreachable() then
    return "Thuê bao tạm thời không liên lạc được"
end if

// Bước 6: Thiết lập kênh thoại hai chiều
establishVoiceChannel()

// Bước 7: Bắt đầu tính cước (billing)
startBilling()

// Bước 8: Chờ kết thúc cuộc gọi
while (callIsActive) do
    if (callEndedByCaller OR callEndedByCallee OR connectionLost) then
        break
    end if
end while

// Bước 9: Kết thúc cuộc gọi
releaseVoiceChannel()
stopBilling()
return "Cuộc gọi kết thúc"

```

Giải thích các bước chính

- **1-2:** Kiểm tra tính hợp lệ của số và điều kiện mạng.
- **3-4:** Thiết lập kết nối qua trạm BTS và tổng đài.
- **5:** Kiểm tra trạng thái máy nhận (bận, tắt, hết vùng phủ).
- **6-7:** Tạo kênh thoại và bắt đầu tính cước.
- **8-9:** Duy trì cuộc gọi cho đến khi một bên kết thúc hoặc mất kết nối.

Câu 7: Phẩm chất của thuật toán và quy trình chuyển đổi sang mã nguồn

Những phẩm chất (tính chất) cần có của một thuật toán

Theo Donald Knuth và các tài liệu kinh điển, một thuật toán tốt cần có các tính chất sau:

1. **Tính hữu hạn (Finiteness):** Thuật toán phải kết thúc sau một số hữu hạn bước.
2. **Tính xác định (Definiteness):** Mỗi bước phải được định nghĩa rõ ràng, không mơ hồ.
3. **Đầu vào (Input):** Có 0 hoặc nhiều đầu vào (giá trị được cung cấp trước khi thuật toán bắt đầu).
4. **Đầu ra (Output):** Có 1 hoặc nhiều đầu ra, là kết quả của thuật toán.
5. **Tính hiệu quả (Effectiveness):** Các bước đủ đơn giản để có thể thực hiện chính xác bằng giấy bút hoặc máy tính trong thời gian chấp nhận được.

Ngoài ra, trong thực tế người ta còn quan tâm đến:

- **Tính đúng đắn (Correctness):** Thuật toán cho kết quả đúng với mọi đầu vào hợp lệ.
- **Tính khả thi (Feasibility):** Có thể cài đặt được trên hệ thống máy tính hiện có.
- **Tính tối ưu (Optimality):** Không có thuật toán nào cùng loại tốt hơn (về thời gian hoặc bộ nhớ).

Các bước chuyển đổi thuật toán thành mã nguồn

1. **Hiểu bài toán:** Xác định rõ đầu vào, đầu ra, ràng buộc.
2. **Lựa chọn kỹ thuật thiết kế:** Tham lam, quy hoạch động, chia để trị, v.v.
3. **Xây dựng thuật toán:** Mô tả ở mức trừu tượng (ngôn ngữ tự nhiên hoặc toán học).
4. **Diễn đạt bằng mã giả:** Trung gian giữa ngôn ngữ tự nhiên và ngôn ngữ lập trình.
5. **Phân tích độ phức tạp:** Đánh giá Big-O về thời gian và bộ nhớ.
6. **Chọn ngôn ngữ lập trình:** Phù hợp với bài toán (Python, Java, C++, ...).
7. **Chuyển mã giả thành mã nguồn:**
 - Dịch các câu lệnh điều kiện, vòng lặp, hàm.
 - Xử lý các cấu trúc dữ liệu (mảng, danh sách, cây, bảng băm).
 - Thêm xử lý ngoại lệ, kiểm tra biên.
8. **Chạy thử với các bộ test:** Đảm bảo thuật toán hoạt động đúng với các trường hợp cơ bản và biên.
9. **Tối ưu hóa (nếu cần):** Cải thiện hiệu năng dựa trên hồ sơ thực thi.
10. **Documentation:** Viết chú thích, tài liệu hướng dẫn sử dụng.

Câu 8: Ví dụ bài toán có nhiều hơn một thuật toán

Bài toán được chọn: Tìm ước chung lớn nhất (GCD) của hai số nguyên dương

Bài toán này có nhiều thuật toán kinh điển. Dưới đây là hai thuật toán tiêu biểu: một đơn giản dễ hiểu, một hiệu quả hơn.

Thuật toán 1: Duyệt trâu bò (Brute-force) – Đơn giản hơn

Ý tưởng

Duyệt từ 1 đến $\min(a, b)$, tìm số lớn nhất chia hết cả a và b .

Mã giả

```
Algorithm GCD_BruteForce(a, b)
Input:  a, b là hai số nguyên dương
Output: gcd(a, b)
```

```
    gcd = 1
    for i = 1 to min(a, b) do
        if (a mod i == 0) and (b mod i == 0) then
            gcd = i
        end if
    end for
    return gcd
```

Độ phức tạp

Thời gian: $O(\min(a, b))$ – chậm khi hai số lớn.

Thuật toán 2: Euclid – Hiệu quả hơn

Ý tưởng

Dựa trên tính chất: $\gcd(a, b) = \gcd(b, a \bmod b)$. Lặp lại cho đến khi số dư bằng 0.

Mã giả

```
Algorithm GCD_Euclid(a, b)
Input:  a, b là hai số nguyên dương
Output: gcd(a, b)
```

```
    while b != 0 do
        temp = b
        b = a mod b
        a = temp
    end while
    return a
```

Ví dụ:

Tìm $\gcd(48, 18)$:

- $(48, 18) \rightarrow (18, 48 \bmod 18 = 12)$
- $(18, 12) \rightarrow (12, 18 \bmod 12 = 6)$
- $(12, 6) \rightarrow (6, 12 \bmod 6 = 0)$
- Kết quả: $\gcd = 6$.

Độ phức tạp

Thời gian: $O(\log \min(a, b))$ – rất nhanh, kể cả với số lớn.

So sánh hai thuật toán

Tiêu chí	Brute-force	Euclid
Độ phức tạp thời gian	$O(\min(a, b))$	$O(\log \min(a, b))$
Dễ hiểu	Rất dễ	Dễ
Hiệu quả với số lớn (10^9)	Rất chậm	Rất nhanh
Số vòng lặp tối đa với $a, b \leq 10^9$	$\approx 10^9$	≈ 30

Kết luận

- **Brute-force:** Đơn giản, dễ cài đặt, phù hợp cho giáo dục và số nhỏ.
- **Euclid:** Hiệu quả vượt trội, được dùng trong mọi thư viện toán học thực tế.

Câu 9: Cải tiến thuật toán tìm khoảng cách nhỏ nhất giữa hai phần tử trong mảng

Thuật toán gốc

Thuật toán được cho có độ phức tạp $O(n^2)$ do hai vòng lặp lồng nhau, xét tất cả các cặp (i, j) kể cả $i = j$ (sau đó loại bằng điều kiện $i \neq j$). Điều này gây lãng phí bộ nhớ và thời gian tính toán.

Các cải tiến đề xuất

1. Giảm số lần so sánh (chỉ xét $i < j$)

Nhận xét: $|A[i] - A[j]| = |A[j] - A[i]|$, do đó chỉ cần xét các cặp với $i < j$.

Algorithm MinDistance_Improved($A[0..n-1]$)

```
d_min
for i      0 to n-2 do
    for j    i+1 to n-1 do
        diff  |A[i] - A[j]|
        if diff < d_min then
```

```

        d_min = diff
    return d_min

```

Cải thiện: Số lần lặp giảm từ n^2 xuống $\frac{n(n-1)}{2}$ (giảm xấp xỉ một nửa).

2. Sắp xếp mảng trước khi tính

Nếu mảng được sắp xếp tăng dần, khoảng cách nhỏ nhất sẽ chỉ xuất hiện giữa các phần tử liên kề.

```

Algorithm MinDistance_Sorted(A[0..n-1])
    Sort(A) // s p x p t n g d n ,           p h c t p O(n log n)
    d_min = A[1] - A[0]
    for i = 0 to n-2 do
        diff = A[i+1] - A[i]
        if diff < d_min then
            d_min = diff
    return d_min

```

Độ phức tạp: $O(n \log n)$, nhanh hơn rõ rệt so với $O(n^2)$ khi n lớn.

3. Dừng sớm nếu tìm được khoảng cách bằng 0

Nếu có hai phần tử bằng nhau, khoảng cách nhỏ nhất là 0, có thể dừng ngay.

```

Algorithm MinDistance_EarlyStop(A[0..n-1])
    d_min = A[1] - A[0]
    for i = 0 to n-2 do
        for j = i+1 to n-1 do
            diff = |A[i] - A[j]|
            if diff < d_min then
                d_min = diff
                if d_min = 0 then
                    return 0
    return d_min

```

Cải thiện: Tối ưu cho trường hợp có phần tử trùng lặp.

4. Sử dụng kỹ thuật chia để trị (Divide and Conquer)

Áp dụng tư tưởng tương tự bài toán tìm cặp điểm gần nhất trong mặt phẳng.

```

Algorithm MinDistance_DAC(A[0..n-1])
    if n <= 1 then return
    if n = 2 then return |A[0] - A[1]|

    mid = n/2
    left_min = MinDistance_DAC(A[0..mid-1])
    right_min = MinDistance_DAC(A[mid..n-1])
    d_min = min(left_min, right_min)

    // X t c c c p q u a b i n g i a h a i n a

```

```
// Ch  c n  x t c c p h n  t  g n  v n g p h n  h o c h
return d_min
```

Độ phức tạp: $O(n \log n)$ trong trường hợp trung bình.

5. Sử dụng cấu trúc bảng băm (Hash Set) nếu biết miền giá trị

Nếu các giá trị trong mảng là số nguyên và biết trước khoảng giá trị, có thể dùng mảng đánh dấu để đạt $O(n)$.

```
Algorithm MinDistance_Hashing(A[0..n-1], maxVal)
    present      m ng boolean k ch t h  c maxVal+1, k h i  t o  fals
    for each x in A do
        if present[x] then
            return 0
        present[x]      true

    d_min
    prev      -1
    for val      0 to maxVal do
        if present[val] then
            if prev      -1 then
                d_min      min(d_min, val - prev)
            prev      val
    return d_min
```

Bảng tổng kết các cải tiến

Phương pháp	Độ phức tạp	Ghi chú
Gốc ($i \neq j$)	$O(n^2)$	Dễ hiểu nhưng chậm
Chỉ xét $i < j$	$O(n^2/2)$	Cải thiện nhẹ
Sắp xếp + duyệt	$O(n \log n)$	Hiệu quả, được khuyến nghị
Dừng sớm khi gặp 0	$O(n^2)$ worst-case	Tốt khi có phần tử trùng
Chia để trị	$O(n \log n)$	Phức tạp hơn
Bảng băm (miền giá trị nhỏ)	$O(n + range)$	Tối ưu khi range nhỏ

Kết luận

Cải tiến tốt nhất và dễ cài đặt nhất là **sắp xếp mảng trước khi tính**, đưa độ phức tạp từ $O(n^2)$ xuống $O(n \log n)$, đồng thời giữ nguyên tính chính xác của thuật toán.

Câu 10: Bản tóm tắt 4 điểm của George Pólya và so sánh với sách giáo khoa

Bản tóm tắt 4 điểm của Pólya (trong sách "How To Solve It")

George Pólya đã đề xuất một quy trình giải quyết vấn đề gồm 4 bước:

1. Hiểu vấn đề (Understand the problem):

- Điều chưa biết là gì? Dữ kiện là gì? Điều kiện là gì?
- Có thể phát biểu lại vấn đề bằng lời của mình không?

2. Xây dựng kế hoạch (Devise a plan):

- Tìm mối liên hệ giữa dữ kiện và ẩn số.
- Có bài toán nào tương tự đã biết không? Có thể áp dụng phương pháp nào (quy nạp, suy luận ngược, chia nhỏ, ...)?

3. Thực hiện kế hoạch (Carry out the plan):

- Thực hiện từng bước một, kiểm tra lại từng bước.
- Có thể chứng minh từng bước là đúng không?

4. Nhìn lại / Đánh giá (Look back):

- Kiểm tra lại kết quả có đúng không?
- Có thể giải bằng cách khác không?
- Có thể áp dụng kết quả cho bài toán khác không?

So sánh với quy trình trong Mục 1.2 của sách giáo khoa

(Lưu ý: Dưới đây là so sánh với quy trình giải quyết vấn đề điển hình trong các sách giáo khoa về thuật toán và lập trình, thường gồm: Phân tích bài toán → Thiết kế thuật toán → Cài đặt → Kiểm thử / Đánh giá.)

Điểm chung

- **Cấu trúc tuần tự:** Cả hai đều chia quá trình giải quyết vấn đề thành các bước tuần tự, có tổ chức.
- **Bước hiểu bài toán:** Cả hai đều nhấn mạnh tầm quan trọng của việc hiểu rõ yêu cầu trước khi bắt tay vào giải.
- **Bước lập kế hoạch / thiết kế:** Đều có bước suy nghĩ về phương pháp, lựa chọn thuật toán hoặc cách tiếp cận.
- **Bước thực hiện / cài đặt:** Đều có bước triển khai cụ thể các ý tưởng đã đề ra.
- **Bước kiểm tra / đánh giá:** Đều có bước xem xét lại kết quả, kiểm thử và cải tiến.

Điểm khác biệt

Tiêu chí	Pólya (How To Solve It)	Sách giáo khoa thuật toán
Mục tiêu chính	Giải quyết vấn đề toán học	Giải quyết vấn đề tính toán / lập trình
Ngôn ngữ	Tổng quát, trừu tượng	Cụ thể, gắn với cấu trúc dữ liệu và mã lệnh
Bước "Look back"	Nhấn mạnh việc học hỏi từ bài toán	Thường chỉ dừng ở kiểm thử (testing)
Tính sáng tạo	Khuyến khích tìm nhiều cách giải	Thường tối ưu theo tiêu chí thời gian/bộ nhớ
Ví dụ minh họa	Bài toán hình học, số học	Sắp xếp, tìm kiếm, đồ thị

Nhận xét bổ sung

- **Pólya** tập trung vào *tư duy giải quyết vấn đề tổng quát*, có thể áp dụng cho toán học, khoa học, kỹ thuật, và cả đời sống. Bước "Look back" của ông rất quan trọng vì nó giúp người học *học cách học* (learn how to learn).
- **Sách giáo khoa thuật toán** thường cụ thể hơn, gắn với các cấu trúc dữ liệu (mảng, danh sách, cây, đồ thị) và các kỹ thuật đánh giá hiệu năng (độ phức tạp Big-O). Bước "kiểm thử" thường chỉ dừng ở việc chạy với các bộ test, ít khi yêu cầu người học suy ngẫm sâu về *tại sao* lời giải đó lại có ý nghĩa.
- **Điểm chung lớn nhất:** Cả hai đều thống nhất rằng **hiểu bài toán** và **kiểm tra lại kết quả** là những bước không thể thiếu trong bất kỳ quá trình giải quyết vấn đề nghiêm túc nào.

Kết luận

Bản tóm tắt 4 điểm của Pólya là một khung tư duy vượt thời gian, có ảnh hưởng sâu sắc đến nhiều lĩnh vực, bao gồm cả khoa học máy tính. Quy trình trong sách giáo khoa thuật toán có thể xem như một trường hợp cụ thể hóa, áp dụng các nguyên lý của Pólya vào bối cảnh thiết kế và phân tích thuật toán.

Bài tập 3: Kiến thức cơ bản về hàm đệ quy nguyên thủy

Các hàm đệ quy nguyên thủy (primitive recursive functions) được xây dựng từ ba hàm cơ bản:

- **Hàm không:** $Z(x) = 0$.
- **Hàm kế vị:** $S(x) = x + 1$.
- **Hàm chiếu:** $P_i^n(x_1, \dots, x_n) = x_i$ với $1 \leq i \leq n$.

Một hàm là đệ quy nguyên thủy nếu nó nhận được từ các hàm cơ bản qua hữu hạn lần áp dụng:

- **Phép hợp thành:** $f(\vec{x}) = g(h_1(\vec{x}), \dots, h_m(\vec{x}))$ với g, h_i đã là đệ quy nguyên thủy.
- **Đệ quy nguyên thủy:**

$$\begin{cases} f(0, \vec{y}) = g(\vec{y}) \\ f(x+1, \vec{y}) = h(x, \vec{y}, f(x, \vec{y})) \end{cases}$$

trong đó g, h là các hàm đệ quy nguyên thủy.

Dưới đây ta sẽ chứng minh lần lượt 12 hàm số là đệ quy nguyên thủy.

1. Phép nhân: $a \times b$

Định nghĩa $\text{mult}(a, b) = a \times b$ bằng đệ quy nguyên thủy theo biến a :

$$\begin{cases} \text{mult}(0, b) = 0 \\ \text{mult}(a + 1, b) = \text{mult}(a, b) + b \end{cases}$$

Ở đây $g(b) = Z(b) = 0$ và $h(a, b, \text{mult}(a, b)) = \text{mult}(a, b) + b$. Vì phép cộng là đệ quy nguyên thủy, nên phép nhân cũng là đệ quy nguyên thủy.

2. Phép lũy thừa: a^b

Định nghĩa $\text{exp}(a, b) = a^b$ bằng đệ quy nguyên thủy theo b :

$$\begin{cases} \text{exp}(a, 0) = 1 \\ \text{exp}(a, b + 1) = \text{exp}(a, b) \times a \end{cases}$$

Trường hợp cơ sở dùng hằng số 1 (có được từ $S(Z(a))$). Vì phép nhân đã là đệ quy nguyên thủy, nên lũy thừa cũng là đệ quy nguyên thủy.

3. Giai thừa: $a!$

Định nghĩa $\text{fact}(a)$:

$$\begin{cases} \text{fact}(0) = 1 \\ \text{fact}(a + 1) = \text{fact}(a) \times (a + 1) \end{cases}$$

Đây là đệ quy nguyên thủy trực tiếp dùng phép nhân và hàm kế vị. Vậy giai thừa là đệ quy nguyên thủy.

4. Hàm trước: $\text{pred}(a)$

$$\text{pred}(a) = \begin{cases} a - 1 & \text{nếu } a > 0 \\ 0 & \text{nếu } a = 0 \end{cases}$$

Định nghĩa:

$$\begin{cases} \text{pred}(0) = 0 \\ \text{pred}(a + 1) = a \end{cases}$$

Ở đây $g() = 0$ và $h(a, \text{pred}(a)) = P_1^2(a, \text{pred}(a)) = a$. Vậy hàm trước là đệ quy nguyên thủy.

5. Phép trừ có kiểm soát: $a \perp b$

$$a \perp b = \begin{cases} a - b & \text{nếu } a \geq b \\ 0 & \text{nếu } a < b \end{cases}$$

Định nghĩa bằng đệ quy nguyên thủy theo a :

$$\begin{cases} 0 \perp b = 0 \\ (a+1) \perp b = \text{pred}(a \perp b) \end{cases}$$

Vì hàm trước pred là đệ quy nguyên thủy, nên phép trừ có kiểm soát cũng là đệ quy nguyên thủy.

6. Hàm $\min(a_1, \dots, a_n)$ (giá trị nhỏ nhất)

Trước hết định nghĩa $\min_2(a, b) = \min(a, b)$:

$$\min_2(a, b) = a \perp (a \perp b)$$

Vì $a \perp (a \perp b)$ chính là $\min(a, b)$. Do $\perp$ là đệ quy nguyên thủy nên $\min_2$ là đệ quy nguyên thủy.

Với $n > 2$:

$$\min(a_1, \dots, a_n) = \min_2(\min(a_1, \dots, a_{n-1}), a_n)$$

Bằng quy nạp, $\min$ là đệ quy nguyên thủy.

7. Hàm $\max(a_1, \dots, a_n)$ (giá trị lớn nhất)

Tương tự, định nghĩa $\max_2(a, b) = \max(a, b)$:

$$\max_2(a, b) = a + (b \perp a)$$

Vì $b \perp a = \max(0, b - a)$ nên $a + (b \perp a) = \max(a, b)$. Vậy $\max_2$ là đệ quy nguyên thủy.

Với $n > 2$:

$$\max(a_1, \dots, a_n) = \max_2(\max(a_1, \dots, a_{n-1}), a_n)$$

Do đó $\max$ là đệ quy nguyên thủy.

8. Khoảng cách tuyệt đối: $|a - b|$

$$|a - b| = (a \perp b) + (b \perp a)$$

Cả hai số hạng đều là phép trừ có kiểm soát, phép cộng là đệ quy nguyên thủy. Vậy $|a - b|$ là đệ quy nguyên thủy.

9. $\sim \text{sg}(a)$: phủ định của hàm dấu (NOT signum)

$$\sim \text{sg}(a) = \begin{cases} 1 & \text{nếu } a = 0 \\ 0 & \text{nếu } a > 0 \end{cases}$$

Có thể định nghĩa:

$$\sim \text{sg}(a) = 1 \perp \text{sg}(a)$$

với sg được định nghĩa ở mục 10. Vì 1 là hằng số và $\perp$ là đệ quy nguyên thủy, nên $\sim \text{sg}$ là đệ quy nguyên thủy nếu sg là đệ quy nguyên thủy.

Cách khác, định nghĩa trực tiếp bằng đệ quy nguyên thủy:

$$\begin{cases} \sim \text{sg}(0) = 1 \\ \sim \text{sg}(a+1) = 0 \end{cases}$$

Với $g() = 1$ và $h(a, \sim \text{sg}(a)) = Z(a) = 0$.

10. $\text{sg}(a)$: hàm dấu (signum)

$$\text{sg}(a) = \begin{cases} 0 & \text{nếu } a = 0 \\ 1 & \text{nếu } a > 0 \end{cases}$$

Định nghĩa:

$$\text{sg}(a) = 1 \perp (1 \perp a)$$

Giải thích: $1 \perp a = 1$ nếu $a = 0$, ngược lại bằng 0. Khi đó $1 \perp (1 \perp a)$ cho kết quả 0 nếu $a = 0$, và 1 nếu $a > 0$.

Vậy sg là đệ quy nguyên thủy (dùng hằng số 1 và phép $\perp$).

11. $a \mid b$: quan hệ chia hết

Định nghĩa hàm đặc trưng $\text{div}(a, b)$ trả về 1 nếu a chia hết b , ngược lại trả về 0:

$$\text{div}(a, b) = \sim \text{sg}(\text{mod}(a, b))$$

trong đó $\text{mod}(a, b)$ là hàm phần dư được định nghĩa ở mục 12. Vì $\sim \text{sg}$ và mod là đệ quy nguyên thủy, nên div cũng là đệ quy nguyên thủy.

12. $\text{MOD}(a, b)$: phần dư của a chia cho b

Định nghĩa $\text{mod}(a, b)$ là phần dư khi chia a cho b , với quy ước $\text{mod}(a, 0) = a$.

Ta dùng đệ quy nguyên thủy theo a :

$$\begin{cases} \text{mod}(0, b) = 0 \\ \text{mod}(a+1, b) = \begin{cases} 0 & \text{nếu } \text{mod}(a, b) + 1 = b \\ \text{mod}(a, b) + 1 & \text{trường hợp còn lại} \end{cases} \end{cases}$$

Viết gọn hơn:

$$\text{mod}(a+1, b) = (\text{mod}(a, b) + 1) \times \sim \text{sg}(|(\text{mod}(a, b) + 1) - b|)$$

Biểu thức này chỉ dùng các hàm đã biết là đệ quy nguyên thủy: cộng, nhân, $\sim \text{sg}$, khoảng cách tuyệt đối, và so sánh với b . Do đó mod là đệ quy nguyên thủy.

Kết luận

Cả 12 hàm số đã được chứng minh là đệ quy nguyên thủy bằng cách xây dựng chúng từ các hàm cơ bản thông qua phép hợp thành và đệ quy nguyên thủy. Các bước quan trọng bao gồm:

- Sử dụng pred để định nghĩa các phép toán dạng trừ.
- Sử dụng $\perp$ để định nghĩa $\min$, $\max$ và khoảng cách tuyệt đối.
- Sử dụng sg và $\sim \text{sg}$ để xử lý các điều kiện logic.
- Sử dụng đệ quy nguyên thủy để định nghĩa phép nhân, lũy thừa, giai thừa và phép chia lấy dư.